# Lecture 6: Assembly I — x86-64 Basics, Registers & Data Movement

Session 7 · Mon Sep 21, 2026

## Contents


Your Program Was Never C . . . 1
Where We Are . . . 2
Why Anyone Reads This . . . 2
One Line, Four Forms . . . 2
The Code We Read Today . . . 3
Run It: Get the Assembly Yourself . . . 3
Where the Registers Actually Are . . . 3
A Register Is a Named Work Slot . . . 4
An Instruction Is a Tiny Command . . . 4
The Sixteen 64-bit General-Purpose Registers . . . 5
The Instruction Vocabulary . . . 5
One Register, Four Names . . . 6
Run It: Writing Through the Narrow Names . . . 6
Two Syntaxes for the Same Bytes . . . 6
`mov` Is a Copy . . . 7
Parentheses: Use the Register Value as an Address . . . 7
Widening: Is `0xFF` 255, or −1? . . . 7
Addressing Modes . . . 8
Run It: One Line of C, One Addressing Mode . . . 8
`lea`: An Address Calculator That Can Do Arithmetic . . . 9
Run It: `lea` Builds Small Multiples . . . 9
Your First Whole Function . . . 9
Two Shifts Right, and Signedness Matters . . . 10
The FLAGS Register . . . 10
`cmp` and `test` Throw the Answer Away . . . 10
A Loop, Line by Line . . . 10
Run It: The Bytes Are Right There . . . 11
The Four Shapes, in a Register . . . 11
AI Systems Connection . . . 11
The Lecture in One Slide . . . 12
Practice . . . 12
Practice — Answers 1–3 . . . 12
Practice — Answers 4–5 . . . 13
Practice — Answers 6–7 . . . 13
Reading . . . 13


## Your Program Was Never C

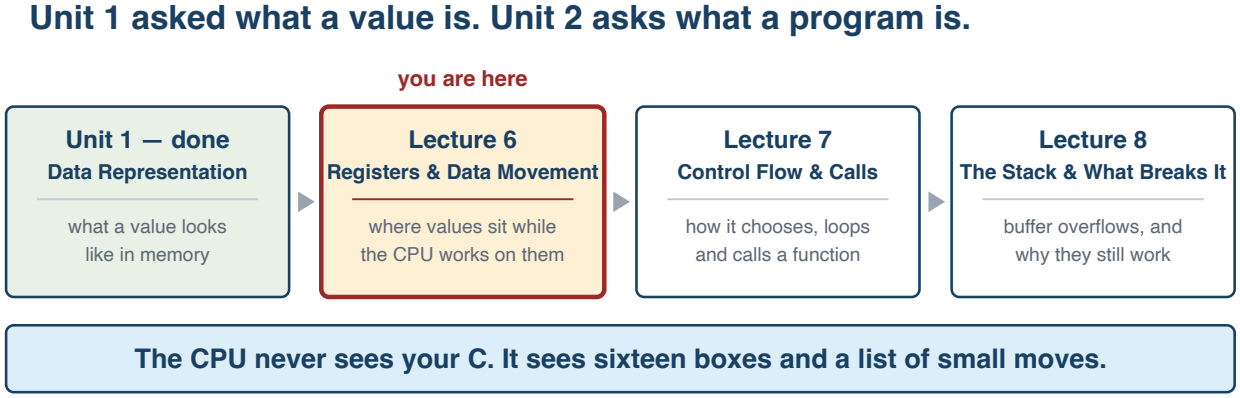
The processor has never seen a `for` loop, a `struct`, or a variable name. By the time your program runs, all of that is gone. What is left is a list of very small instructions over sixteen boxes:

```
total += arr[i];        // what you wrote
```

```
addq (%rdi), %rax       # what runs
```

Unit 1 asked what a **value** looks like in memory. Unit 2 asks what a **program** looks like. Same question, other half of the machine.

## Where We Are



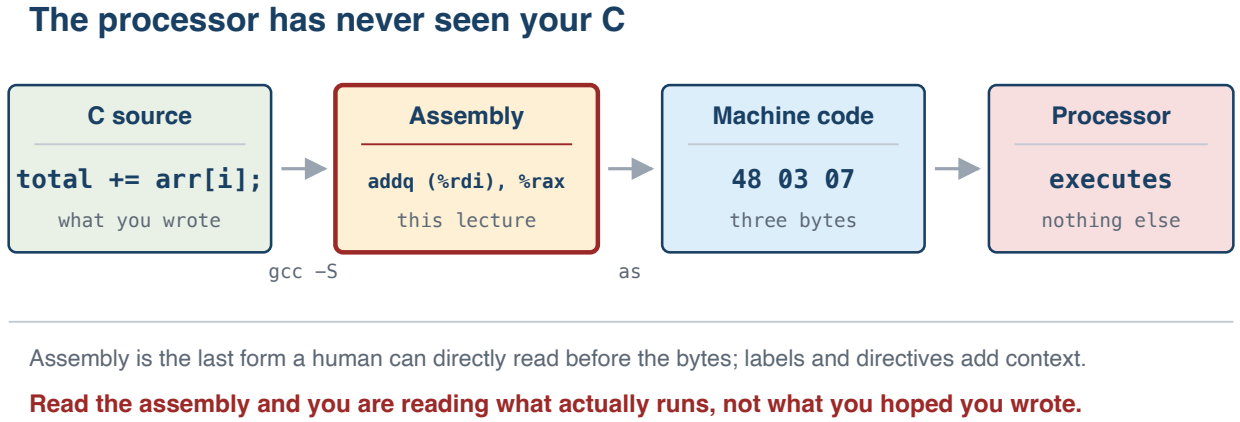

Three lectures. Today is the vocabulary: where values sit, and how they move. We will name branches and calls today, but Lecture 7 studies how control flow and calls work.

## Why Anyone Reads This

You will not write assembly for a living. You will read it — when the crash report has no source, when the profiler blames a line that looks free, or when the optimizer deletes a check and you want to know why.

It is also the only honest answer to *"is this actually faster?"* The source code is a proposal. The assembly is what the machine agreed to.

## One Line, Four Forms




Assembly is a human-readable notation for machine instructions. Most of the time, one instruction has one line of assembly and a corresponding sequence of bytes; labels, directives, and relocations are bookkeeping around it.

## The Code We Read Today

All of `01-first-look.c` except its `main`. Four functions, and each one comes back later in the lecture:

```c
int  add(int a, int b)                 { return a + b; }
long triple(long x)                    { return x * 3; }
long index_it(const long *arr, long i) { return arr[i]; }

long sum(const long *arr, long n) {
    long total = 0;
    for (long i = 0; i < n; i++) total += arr[i];
    return total;
}
```

Nothing here is clever. That is the point — you already know exactly what each one does, so anything surprising in the assembly is the *machine*, not the algorithm.

## Run It: Get the Assembly Yourself

No debugger needed. In `sample code/lecture06/`, ask the supplied build for x86-64 assembly — `-S` means "stop after assembly":

```
make asm
```

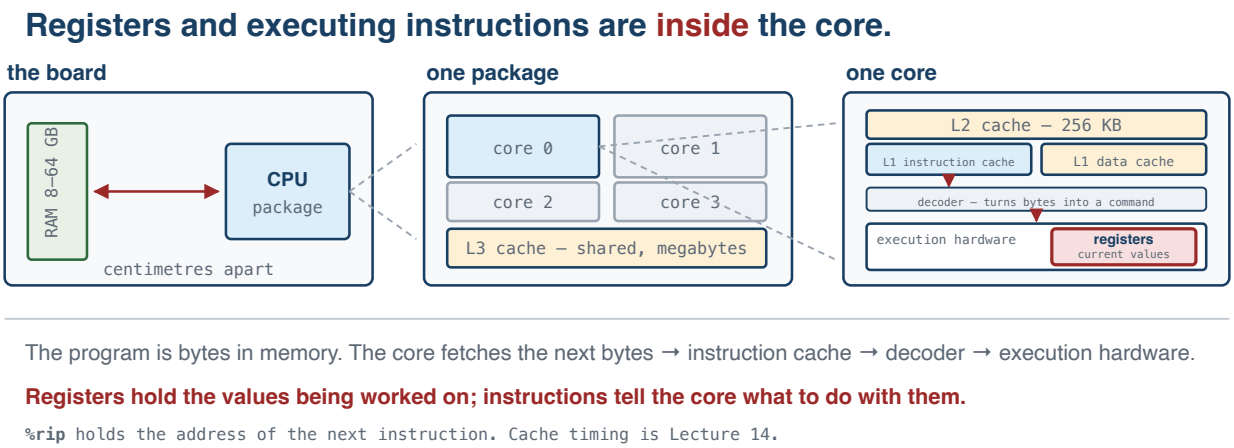
The first thing in `01-first-look.s`:

```
_add:
    leal    (%rdi,%rsi), %eax
    retq
```

That is `add`, whole. Two instructions, and neither one is `add`.

*The exact labels and spacing are compiler- and platform-specific. The provided `make asm` selects x86-64 on Apple Silicon too.*

## Where the Registers Actually Are




The program is bytes in memory. The core fetches the next bytes → instruction cache → decoder → execution hardware.

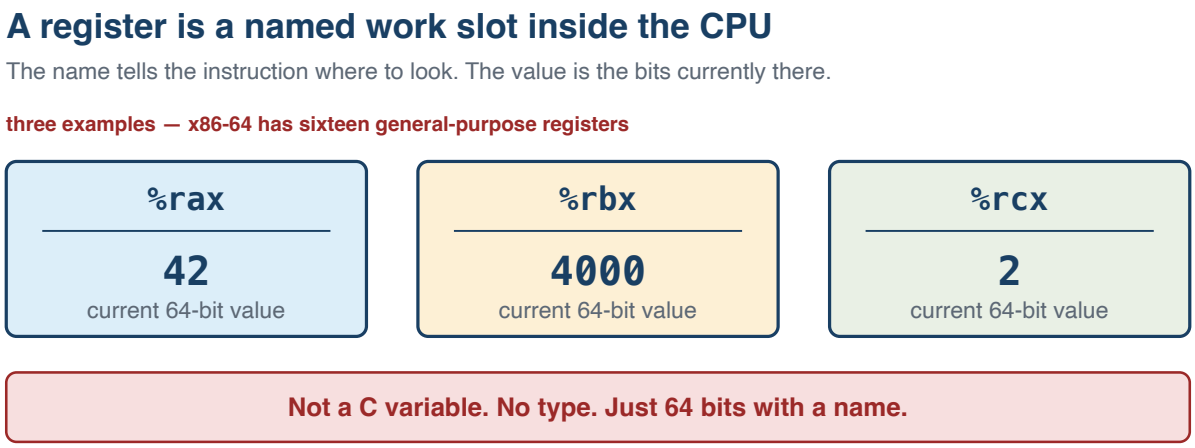
**Registers hold the values being worked on; instructions tell the core what to do with them.**

`%rip` holds the address of the next instruction. Cache timing is Lecture 14.

Lecture 2 opened with a photograph of a RAM module — memory you can hold. The registers are not on that stick, and they are **not the cache** either: their storage is inside the core, close to the hardware that does arithmetic.

Instructions start as program bytes in memory. To run one, the core fetches its bytes through the instruction cache, decodes them into a command, then sends that command to execution hardware. `%rip` names the address of the next bytes to fetch.

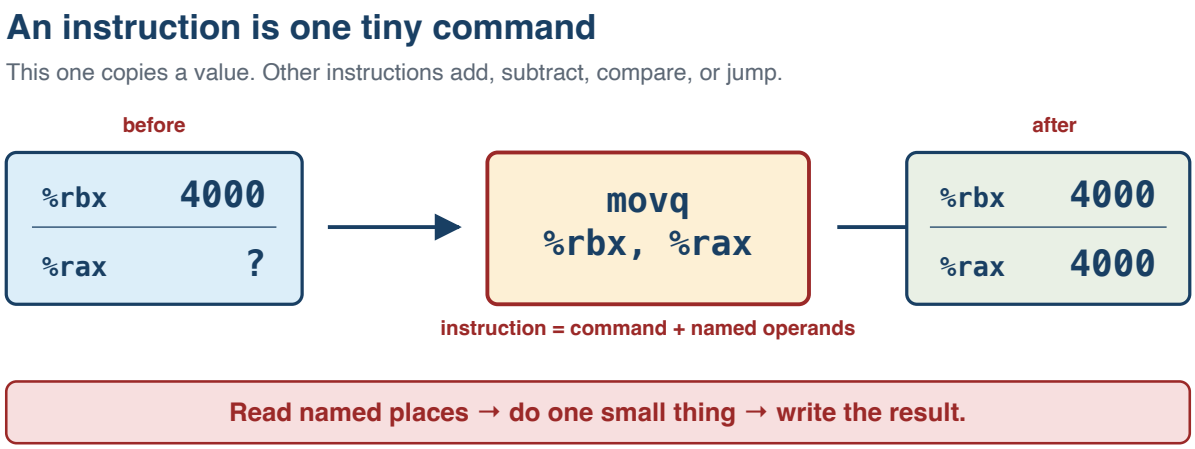
## A Register Is a Named Work Slot




A **register** is a small, named place inside the CPU that holds a value it is working with right now. Think of `%rax` as the name written on one work slot, not as a C variable: it has no type, only 64 bits.

This machine has sixteen general-purpose slots. We will use a few before we memorize their conventional jobs.

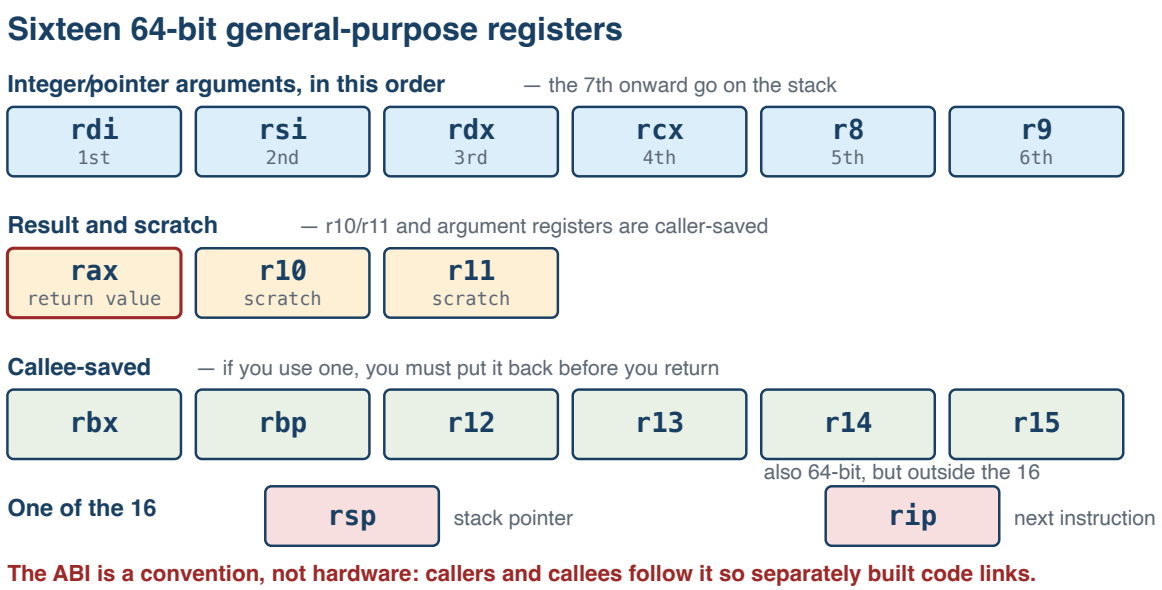
## An Instruction Is a Tiny Command




An **instruction** is one tiny command. `movq %rbx, %rax` means: copy the 64-bit value in `%rbx` into `%rax`. `movq` is the command; `%rbx, %rax` name the slots it reads and writes.

That is the pattern for every assembly line: read named places, do one small thing, write a result somewhere.

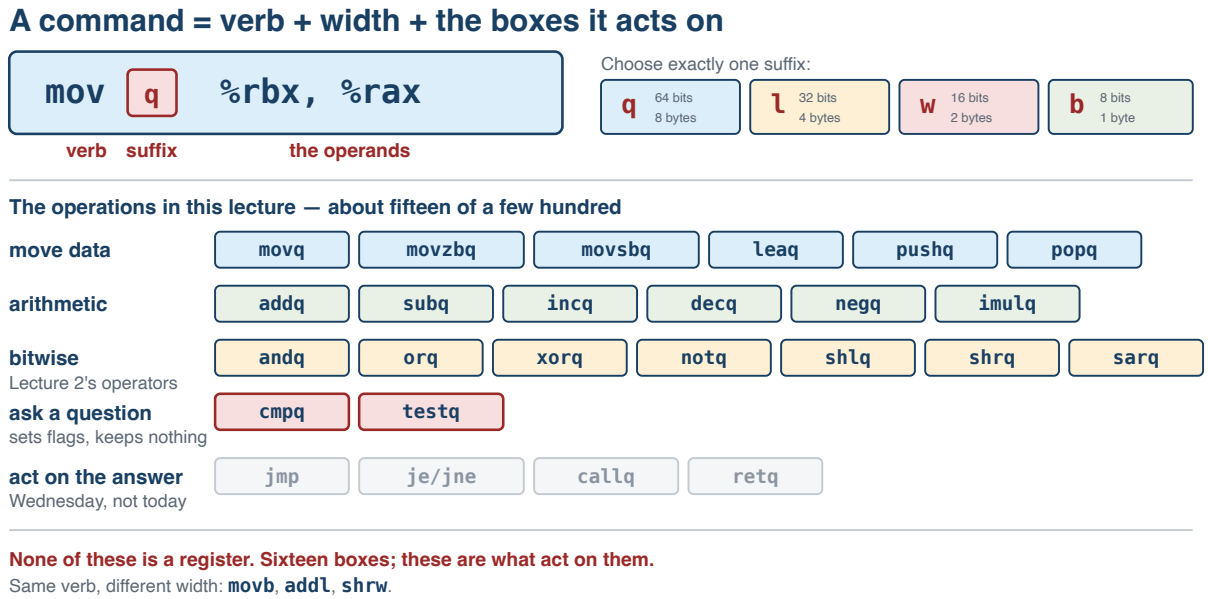
## The Sixteen 64-bit General-Purpose Registers




All 16 general-purpose registers here are **64 bits**: `%rax`, `%rdi`, `%r8`, and `%rsp` are the same width. `%rip` is also 64 bits, but it is a special instruction pointer, **not** one of the 16 general-purpose registers.

The shorter aliases follow the same idea: `%rbx` → `%ebx` → `%bx` → `%bl`, and `%r8` → `%r8d` → `%r8w` → `%r8b`. The colors show **System V x86-64 ABI** jobs, an agreement between caller and callee — not hardware rules.

## The Instruction Vocabulary




Read `movq %rbx, %rax` left to right: `mov` is the verb, `q` chooses **64 bits (8 bytes)**, and the operands name the slots. Choose one suffix: `q` = 64/8, `l` = 32/4, `w` = 16/2, `b` = 8/1. The rows below are this lecture's commands; jumps and calls are Wednesday.

## One Register, Four Names




Start with the count: `%rax` is one of the 16. `%eax`, `%ax`, and `%al` are not extra entries; they name its low 32, 16, and 8 bits. The names are history: 16-bit x86 had `ax`; its 32-bit extension was `eax`; x86-64 added `rax`. Hardware keeps the old names so older code still works.

**The rule that catches people:** writing `eax` zeroes the top half. Writing `ax` or `al` does not.

## Run It: Writing Through the Narrow Names

```
# start with   %rax = 0xAAAAAAAABBBBBBBB
# then write 0x11 through each name:

movb  $0x11, %al      # 0xAAAAAAAABBBBBB11
movw  $0x11, %ax      # 0xAAAAAAAABBBB0011
movl  $0x11, %eax     # 0x0000000000000011   <- top half wiped
```

**What each write changes:**

- Write `%al`: replace bits 0–7; leave the other 56 bits alone.
- Write `%ax`: replace bits 0–15; leave the other 48 bits alone.
- Write `%eax`: replace bits 0–31, then the hardware sets bits 32–63 to zero.

**Why `xorl %eax, %eax` appears everywhere:**

- `x ^ x` is zero, so it writes zero to `%eax`.
- The special 32-bit rule then makes **all of `%rax` zero**.
- It takes 2 bytes; `movq $0, %rax` takes 7.

## Two Syntaxes for the Same Bytes

The same instruction is written two ways, and you will meet both:

| AT&T — **we use this** | Intel — Intel’s manuals, online docs |
|---|---|
| `movq %rax, %rbx` | `mov rbx, rax` |
| `addq $8, %rsp` | `add rsp, 8` |
| `movq 8(%rbx), %rax` | `mov rax, [rbx + 8]` |

**We use AT&T** — what CS:APP prints and what `gcc`, `gdb` and `objdump` give you by default. **Source first**: destination on the right, `%` on registers, `$` on constants, size as a suffix.

**Intel reverses the operands.** No `%` signs means you are reading Intel.

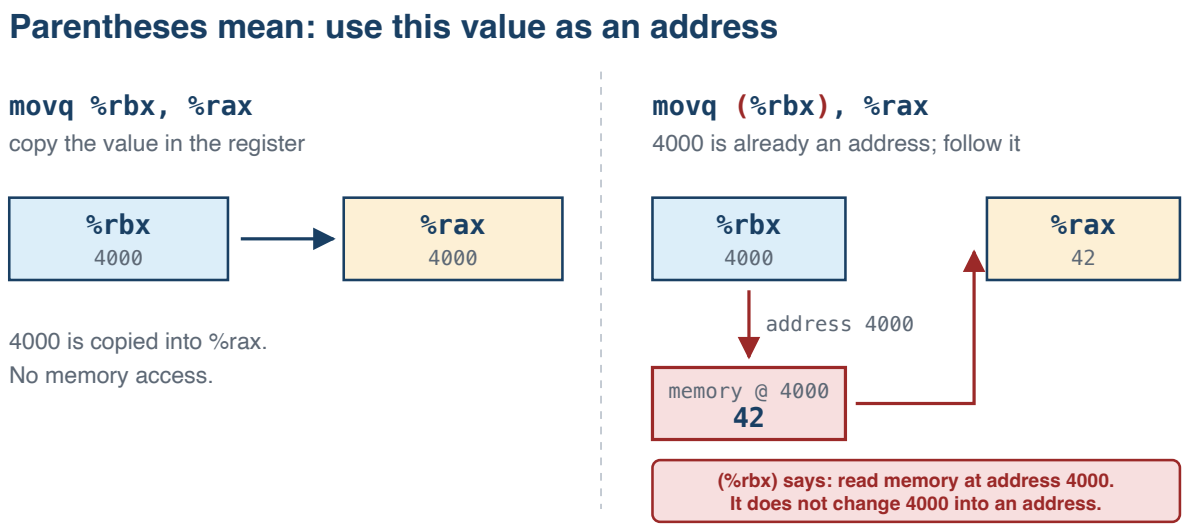
## `mov` Is a Copy

```
movq  %rbx, %rax     # rax = rbx       register -> register
movq  (%rbx), %rax   # rax = *rbx      memory   -> register
movq  %rax, (%rbx)   # *rbx = rax      register -> memory
movq  $42, %rax      # rax = 42        constant -> register
```

**Source first, destination last** — it reads left to right, like an arrow. `movq %rbx, %rax` overwrites `rax` and leaves `rbx` alone: it copies, despite the name.

One combination is missing on purpose: there is no `movq (%rax), (%rbx)`. Memory to memory takes two instructions, because it takes two trips.

## Parentheses: Use the Register Value as an Address




Here, `%rbx` holds the pointer value 4000. Parentheses do **not** convert it into an address; they tell `movq` to dereference it: read memory at address 4000. If a C pointer `p` currently lives in `%rbx`, then `(%rbx)` corresponds to `*p`.

## Widening: Is `0xFF` 255, or −1?

Because the operation carries the size, writing a constant into memory is never ambiguous — `movq $42, (%rbx)` writes eight bytes, `movb $42, (%rbx)` writes one.

Going the other way, from a narrow value to a wide one, needs a **decision**. Suppose the low byte contains `0xFF`:

```
%al = 0xFF          11111111
```

Those eight bits do not carry their own type. If they are an **unsigned** byte, `11111111` means 255. If they are a **signed** byte, the top bit is the sign bit, so they mean −1. Widening must preserve the interpretation:

```
movzbq  %al, %rax   00000000000000FF   zero-extend: 255
movsbq  %al, %rax   FFFFFFFFFFFFFFFF   sign-extend: -1
```

`movzbq` fills the new upper bits with zero. `movsbq` copies the old sign bit (1) into every new upper bit. The source byte is the same; only the widening rule differs. The instruction does not discover whether C called the value `unsigned char` or `signed char` — the compiler chooses the instruction from that type.

## Addressing Modes

### One address form covers almost every array and struct access

| Disp( | Base, | Index, | Scale ) |
|---|---|---|---|
| a constant<br>fixed offset | a register<br>where it starts | a register<br>which element | 1, 2, 4 or 8<br>size of one element |

**All four at once — an array of 8-byte structs:**

`struct P { int x, y; } *a; a[i].y` → `movl 4(%rdi,%rsi,8), %eax`

**4** = Disp · **%rdi** = Base · **%rsi** = Index · **8** = Scale · y sits 4 bytes into each struct

**Leave a part out and it takes its default:**

| | | |
|---|---|---|
| `(%rdi,%rsi,8)` | = `%rdi + %rsi*8` | no Disp — it is 0 |
| `8(%rdi)` | = `%rdi + 8` | no Index, no Scale |
| `(%rdi,%rsi)` | = `%rdi + %rsi` | no Scale — it is 1 |
| `(%rdi)` | = `%rdi` | just `*p` — a plain dereference |

**An omitted Disp is 0. An omitted Scale is 1. Nothing is ever left undefined.**

`Base + Index x Scale + Disp`, where `Scale` is 1, 2, 4 or 8 — the sizes things actually are.

**Parts you leave out are not missing, they are defaulted:** no `Disp` means 0, no `Scale` means 1. That is why `(%rdi,%rsi,8)` is a complete address even though the form has four slots and you can only see three.

## Run It: One Line of C, One Addressing Mode

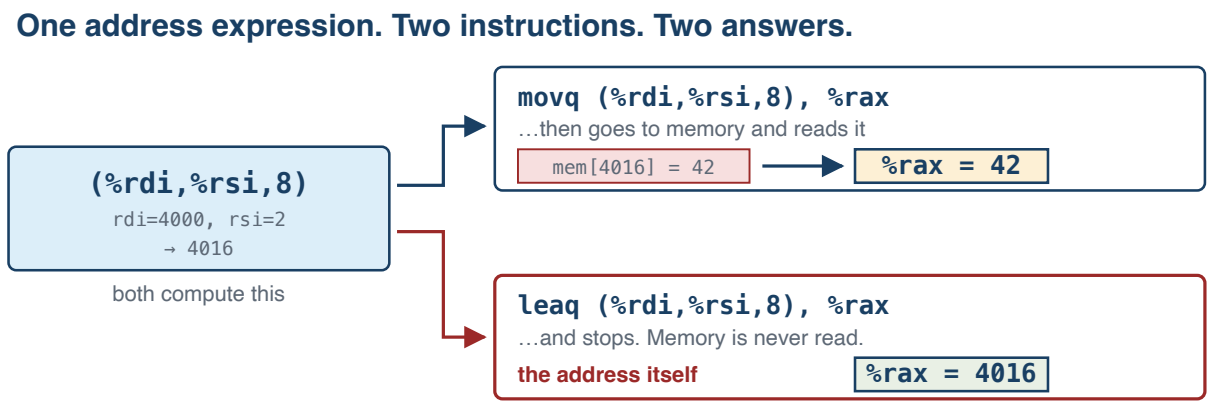
```c
long index_it(const long *arr, long i) {
    return arr[i];
}
```

```
_index_it:
    movq    (%rdi,%rsi,8), %rax
    retq
```

`arr` arrived in `%rdi`, `i` in `%rsi`, the answer leaves in `%rax`, and a `long` is 8 bytes so the scale is 8. The entire function body is one addressing mode.

Read `(%rdi,%rsi,8)` as *base, index, scale* — exactly the order CS:APP writes it in.

## `lea`: An Address Calculator That Can Do Arithmetic




So **`leaq`** is used as plain arithmetic. A different example:
**`leaq (%rdi,%rdi,2), %rax`** computes **`rax = rdi * 3`** — one instruction, no memory, no multiplier.

`lea` — **L**oad **E**ffective **A**ddress — evaluates the address formula but stops before the memory access:

```
D(B,I,S)  means  D + B + I × S
```

With `leaq`, the result goes into a register. No memory is read, and flags are unchanged. That makes the addressing formula useful as a small arithmetic calculator even when no pointer is involved.

## Run It: `lea` Builds Small Multiples

```
long times3(long x) { return x * 3; }
long times7(long x) { return x * 7; }
```

```
leaq  (%rdi,%rdi),   %rax    # rax = x + x     = 2x
leaq  (%rdi,%rdi,2), %rax    # rax = x + 2x    = 3x
leaq  (%rdi,%rdi,4), %rax    # rax = x + 4x    = 5x
leaq  (%rdi,%rdi,8), %rax    # rax = x + 8x    = 9x
leaq  7(,%rdi,8),    %rax    # rax = 7 + 8x
```

The `3x` line is the one to remember: the base contributes `x`, and the index contributes `2x`. The scale is not an arbitrary multiplier; it can only be 1, 2, 4, or 8. So `7x` needs two instructions, for example `8x - x`:

```
leaq  (,%rdi,8), %rax    # rax = 8x
subq  %rdi, %rax          # rax = 8x - x = 7x
```

When you see `lea` in compiler output, first expand the address formula. Then ask whether it is being used for an address or for arithmetic.

## Your First Whole Function

The two lines from the start of the lecture:

```
int add(int a, int b) { return a + b; }
```

```
_add:
    leal    (%rdi,%rsi), %eax
    retq
```

Read it off the conventions: first argument in `rdi`, second in `rsi`, return value in `rax` — `eax` here because the result is an `int`. And the addition is done by `lea`, not `add`, because that way the answer lands straight in `eax`

without disturbing either input.

Nothing else happens. No stack, no setup. This is what "the compiler is good at this" looks like.

## Two Shifts Right, and Signedness Matters

The operations map had `shrq` and `sarq` side by side. They are not interchangeable — they differ in what gets shifted *in* at the top:

```
shrb  $1, %al      # fills with zeros      -> for unsigned bytes
sarb  $1, %al      # fills with the sign   -> for signed bytes
```

For an 8-bit value, shift `0xF0` right by 4 and you get `0x0F` with `shrb` but `0xFF` with `sarb`. Same eight bits, same instruction family, opposite answers.

This is Lecture 2's arithmetic-versus-logical shift. The C type is one input to the compiler's choice, along with the operation and the surrounding code; it is not information stored in the bits themselves.

## The FLAGS Register

`add` and `sub`-style arithmetic instructions quietly write down four facts about their result:

| Flag | Set when |
|---|---|
| `ZF` | the result was zero |
| `SF` | the result was negative (top bit set) |
| `CF` | carry out for addition, or borrow for subtraction — unsigned overflow |
| `OF` | it overflowed — **signed** overflow |

Lecture 5 said unsigned and signed overflow are different questions. Here they are, as two different bits, recorded at the same time by the same instruction. `test` sets `CF` and `OF` to zero; later flag-setting instructions overwrite the old flags. Nothing decides which one mattered until something reads them.

## `cmp` and `test` Throw the Answer Away

There is no "compare" circuit. `cmp` is a subtraction that discards its result and keeps only the flags:

```
cmpq  %rbx, %rax    # compute rax - rbx, set flags, keep nothing
testq %rax, %rax    # compute rax & rax, set flags, keep nothing
```

> **Warning**
>
> Read `cmp` carefully: `cmpq %rbx, %rax` computes **`rax - rbx`** — the *second* operand minus the first. Source-first bites hardest here, because the subtraction reads backwards from the line.

So it sets `ZF` exactly when `rax == rbx`, and `testq %rax, %rax` sets `ZF` exactly when `rax` is zero — the cheapest way to ask "is this zero or NULL?".

The flags sit there until something acts on them. That is Wednesday.

## A Loop, Line by Line

For `long sum(const long *arr, long n)`, `arr` arrives in `%rdi` and `n` in `%rsi`:

```
_sum:
    testq %rsi, %rsi            # n <= 0?
    jle   .empty                # return 0
    xorl  %ecx, %ecx            # i = 0
    xorl  %eax, %eax            # total = 0
.loop:
```

```
    addq  (%rdi,%rcx,8), %rax     # total += arr[i]
    incq  %rcx                    # i++
    cmpq  %rcx, %rsi              # n - i
    jne   .loop                   # i != n
    retq
.empty:
    xorl  %eax, %eax              # return 0
    retq
```

**Note**

Every piece is from today; only the jumps are new. The compiler checks `n <= 0` first, so this `for` becomes a **guard** plus a **do-while**.

## Run It: The Bytes Are Right There

```
make dis
cat 01-first-look.dis
```

```
0000000000000000 <_add>:
       0: 8d 04 37         leal  (%rdi,%rsi), %eax
       3: c3               retq

0000000000000020 <_index_it>:
      20: 48 8b 04 f7      movq  (%rdi,%rsi,8), %rax
      24: c3               retq
```

Three bytes for the `lea`, four for the `mov`, one for `ret`. The `48` in front is a REX prefix with its `W` bit set, selecting a 64-bit operation for this instruction. In this example, that makes the `rax` version one byte longer than the corresponding `eax` version.

## The Four Shapes, in a Register

Lecture 5b ended by promising you would recognise its four shapes in a register dump. A register is 8 bytes; here they are:

```
  int 42                4 bytes   0x000000000000002A
  long -1               8 bytes   0xFFFFFFFFFFFFFFFF
  int * (an address)    8 bytes   0x000000016F8467B8
  float 3.0f            4 bytes   0x0000000040400000
  double 3.0            8 bytes   0x4008000000000000
  struct point{1,2}     8 bytes   0x0000000200000001
```

`0x40400000` is the `3.0f` you decoded on Tuesday. For `struct point { int x; int y; } = {1,2}`, the numeric 64-bit display has `y` in the high 32 bits and `x` in the low 32; its memory bytes begin with `x`, because the machine is little-endian.

**Nothing in a general-purpose register says which shape it holds.** A GPR may hold integer data, an address, or bits being reinterpreted. A memory operand uses a computed address to access data; the instruction determines how those loaded or stored bits are interpreted. Floating-point instructions such as `addss` use XMM registers instead.

## AI Systems Connection

The kernels that dominate a training profile — matmul, attention, softmax — are hand-tuned, and the tuning is done by reading exactly what you read today. A register spill in an inner loop shows up as an unexpected `mov` to `N(%rsp)`. That is how you find it.

The other half is security. The ML stack is full of C parsers — ONNX, safetensors, image decoders — and a malicious model file need not break the math. It can corrupt the program instead.

Lecture 8 is that attack in full. Today you learned to read the ammunition.

## The Lecture in One Slide

| | what it is | the thing to remember |
|---|---|---|
| registers | 16 general-purpose boxes | System V integer/pointer args arrive in `rdi, rsi, rdx, rcx, r8, r9` |
| `rax` / `eax` | one register, two widths | writing `eax` zeroes the top half |
| `mov` | a copy | destination first; no memory-to-memory |
| `(...)` | go to that address | it is Lecture 3's ∗, spelled differently |
| `lea` | the same arithmetic, no memory | it is a multiplier, not a pointer thing |
| flags | facts about the last result | `cmp` and `test` set them and keep nothing else |

Next: **Control Flow & Calls** — the jumps that read those flags, and what a function call actually costs.

## Practice

1. Write a one-line C function and get its assembly with `cc -O1 -S`. Identify which register each argument arrived in.
2. What is the difference between `movq %rbx, %rax`, `movq (%rbx), %rax`, and `leaq (%rbx), %rax`? Give the value in `%rax` for each, if `%rbx` holds 4000 and memory at 4000 holds 42.
3. `leaq (%rdi,%rdi,4), %rax` — what does this compute? Write the C.
4. Why can the compiler use `lea` for `x * 5` but not for `x * 7` in one instruction? What does it do instead?
5. A function starts with `testq %rdi, %rdi`. What is the C almost certainly checking?
6. Write `long get(struct s *p)` returning a field 16 bytes into the struct. Predict the addressing mode before you compile it, then check.
7. Take `sum` from the slides and change `long` to `int`. Which registers change name, and why does the addressing mode's scale change?

## Practice — Answers 1–3

**1.** Any one-liner works. The first argument arrives in `%rdi`, the second in `%rsi`, and the result leaves in `%rax` — or `%eax` when it is an `int`.

**2.** With `%rbx` = 4000 and memory at 4000 holding 42:

| | `%rax` becomes | why |
|---|---|---|
| `movq %rbx, %rax` | 4000 | copies the value |
| `movq (%rbx), %rax` | 42 | goes to that address and reads it |
| `leaq (%rbx), %rax` | 4000 | computes the address, never reads |

The first and third agree here — which is the point. `leaq` only earns its keep when there is arithmetic inside the parentheses.

**3.** `(%rdi,%rdi,4)` is `rdi + rdi*4`, so `5*rdi`. The C is `return x * 5;`

## Practice — Answers 4–5

**4.** 5 is `1 + 4` — exactly `Base + Index × Scale` with a scale of 4, so one `leaq` reaches it. 7 is not `1 + {1,2,4,8}`, so no single address form can. The compiler computes `8x - x` instead:

```
leaq (,%rdi,8), %rax
subq %rdi, %rax
```

Two instructions, and still no multiply.

**5.** `testq %rdi, %rdi` sets `ZF` exactly when `%rdi` is zero, and `%rdi` is the **first argument**. So the C is checking that argument against zero — `if (!p)` for a pointer, or `if (n == 0)` for a count.

## Practice — Answers 6–7

**6.** `movq 16(%rdi), %rax` — Base plus Disp, with **no Index and no Scale**, because the offset is a compile-time constant. Nothing has to be multiplied.

**7.** `sizeof(int)` is 4, so the scale drops from 8 to 4. A typical loop keeps the pointer in `%rdi`, the count in `%esi`, the index in a 64-bit register, and the running total in `%eax`:

```
addl  (%rdi,%rcx,4), %eax      # total += arr[i]
```

The *shape* of the loop is unchanged — same guard, same backward jump, same addressing mode. The element width changes the scale and the arithmetic width; the compiler is still free to choose its scratch registers.

## Reading

- CS:APP §3.1–3.3 (a historical perspective, encodings, data formats)
- CS:APP §3.4 (accessing information — registers and operand forms)
- CS:APP §3.5 (arithmetic and logical operations)

CS:APP and these slides use **AT&T** syntax: source first, destination last.